

Unidad 9

Arquitectura evolutiva

El sistema que construimos hoy va a vivir en un mundo que todavía no existe. Cambian los requisitos, el negocio, las herramientas y el equipo.



¿Qué es arquitectura evolutiva?

"Una arquitectura evolutiva soporta el cambio guiado e incremental a través de múltiples dimensiones." Ford, Parsons y Kua.

La arquitectura no es una fase que termina cuando parte el desarrollo. Las decisiones que tomamos al inicio van quedando obsoletas.

En vez de resistir el cambio, diseñamos para que cambiar sea barato.

Dos ideas centrales en la definición:

- Guiado: el cambio se evalúa contra criterios explícitos y verificables, no contra la opinión del arquitecto de turno.
- Incremental: se avanza en pasos pequeños y reversibles, no en reescrituras heroicas.



Funciones de aptitud

Una función de aptitud (*fitness function*) es una evaluación, idealmente automatizada, que mide qué tan bien la arquitectura cumple una característica que nos importa: latencia, acoplamiento, seguridad, costo.

Es el mecanismo que guía la evolución.

Si una característica es importante, debe tener una función de aptitud.

Si no la tiene, se va a degradar sin que nadie se dé cuenta.

Ejemplo

Queremos que ningún servicio acceda a la base de datos de otro (**Unidad 2**). Una prueba en el CI lo verifica:

```
def test_no_acopla_bases_de_datos():  
    for archivo in Path("service_01/app").glob("**/*.py"):  
        assert "teams_db" not in archivo.read_text()
```

Si alguien agrega la conexión prohibida, el *pipeline* falla.

La regla de arquitectura dejó de ser un documento que nadie lee y pasó a ser una prueba que se ejecuta en cada *commit*.

Tipos de funciones de aptitud

- Atómicas: miden una característica en un contexto acotado, como la prueba anterior.
- Holísticas: miden varias características combinadas, como una prueba de carga sobre datos cifrados.

- Activadas: corren en el CI o de forma programada.
- Continuas: corren todo el tiempo en producción, como el *logging* de la **Unidad 1** o Chaos Monkey de la **Unidad 8**.



Cambio guiado e incremental

El cambio incremental tiene dos caras:

- Cómo se desarrolla: cambios pequeños, integrados con frecuencia, cada uno validado por las funciones de aptitud.
- Cómo se despliega: entregas graduales y reversibles, como los despliegues progresivos de la **Unidad 5**.

Un cambio grande es una apuesta. Muchos cambios chicos, cada uno verificado, son un experimento controlado.

Si algo sale mal, se revierte un paso y no seis meses de trabajo.

Acoplamiento apropiado

Arquitectura evolutiva no es cero acoplamiento, es acoplamiento apropiado: acoplar lo que cambia junto y desacoplar lo que cambia por separado.

Ya conocemos las herramientas: información oculta (**Unidad 2**), contratos explícitos y versionamiento (**Unidades 3 y 4**).

Lo que evoluciona con libertad es lo que está detrás de una frontera bien definida.



Deuda arquitectónica

Es la deuda técnica a nivel de estructura: decisiones que fueron correctas y dejaron de serlo, o atajos que acoplaron lo que debía estar separado.

Se paga con interés: cada cambio siguiente cuesta más.

Las funciones de aptitud detectan la deuda antes de que duela.

El cambio incremental permite pagarla de a poco, sin detener la entrega de valor.



Microservicios y evolución

Todo lo que vimos en el curso habilita la evolución:

- Despliegue independiente (**Unidades 1 y 5**): cambios pequeños y reversibles, servicio por servicio.
- Fronteras según el dominio (**Unidad 2**): el cambio queda contenido en un servicio.

- Contratos y versionamiento (**Unidades 3 y 4**): evolucionar sin romper a los consumidores.
- Pruebas de servicio y de contrato (**Unidad 7**): funciones de aptitud del sistema.
- Resiliencia y escalabilidad (**Unidad 8**): características protegidas con funciones de aptitud continuas.

Por eso microservicios es el ejemplo favorito de los autores: es una arquitectura con muchas dimensiones de cambio ya desacopladas.

El costo es la complejidad operacional que pagamos durante todo el curso. La evolución no sale gratis, se compra.



Tarea

Define tres funciones de aptitud para el sistema del trabajo final: una de acoplamiento, una de rendimiento y una a elección. Implementa al menos una de forma automatizada en el CI de tu microservicio. Documenta las otras dos: qué mide cada una, su umbral y dónde correría.



Material complementario

- Building Evolutionary Architectures: Automated Software Governance, 2a edición. Neal Ford, Rebecca Parsons, Patrick Kua y Pramod Sadalage (2022). O'Reilly.
- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capítulo 16.